GROSS DOMESTIC PRODUCT ANALYSIS

USING PYTHON

ANUJ BURDE

KAUSTUBH NAYAK

SALONI BERA

RAJ VERMA

ABSTRACT

GDP Analysis is a key measure of a country's economic performance, it is also important to understand and analyze the pattern and its trend to make decisions with technology advancing every moment the country economy is no exception, it involves analyzing the data, managing the data, visualizing and generating useful insights. Data science is becoming one of the most widely used technology at such a high speed and it is helping the government in maintaining data and transparency, becoming efficient and also assisting to boost the economy and productivity. Using statistics, data preparation, predictive modeling, and machine learning, it attempts to resolve various problems within different areas and the economy on the whole. Machine learning can be a powerful tool for GDP analysis by using statistical model to forecast businesses. Ultimately machine learning has the potential to provide valuable insights and also analyze the impact on GDP analysis Overall, the objective of GDP analysis using machine learning is to provide stakeholders with valuable insights into economic performance, enabling them to make informed decisions about economic growth and development

INDEX

INTRODUCTION

Gross Domestic Product (GDP) analysis is a method of examining the performance of an economy by analyzing the total value of goods and services produced within a country's borders. GDP is a key economic indicator used to measure the overall economic health of a country, as well as to compare economic performance across different countries and time periods. GDP analysis can be performed using a variety of statistical tools and techniques, including regression analysis, time series analysis, and data visualization. Python is a popular programming language for GDP analysis due to its flexibility, simplicity, and powerful data analysis libraries such as pandas, NumPy, and matplotlib

GDP per capita = Gross Domestic Product (GDP) / Population

Gathering data: The first step in GDP analysis is to gather data on the country's economic activity, such as output, consumption, investment, government spending, and net exports. This data is typically obtained from national statistical agencies or international organizations such as the World Bank or the International Monetary Fund.

Calculating GDP: Once the data has been gathered, the next step is to calculate the country's GDP. This is typically done by adding up the value of all final goods and services produced within the country during a given time period, usually a year or a quarter.

Analyzing trends: After calculating GDP, analysts can then analyze trends in economic growth over time. This may involve examining changes in GDP from year to year, as well as changes in the composition of GDP (changes in the relative importance of different sectors of the economy

OBJECTIVE

Analysis of the world GDP dataset and getting insights and understanding the factors effecting and contributing towards the GDP of each country also

Using machine learning models to predict and identify key economic builders

METHODS USED AND IMPLIMEMENTATION

Python is a popular programming language for data analysis due to its simplicity, flexibility, and powerful libraries. Here are the steps to perform GDP analysis using Python:

Data collection: Collect data on GDP from reputable sources such as the World Bank, IMF, or national statistical agencies. The data should include the GDP values for each year, population, and other relevant indicators such as inflation rates, exports, imports, and unemployment rates.

Data preparation: Load the data into Python using pandas, a popular data manipulation library. Clean and transform the data as necessary, removing missing or inconsistent data, and converting the data into a suitable format for analysis.

Data visualization: Visualize the data using Python libraries such as matplotlib or seaborn. Create line graphs, bar charts, or other suitable visualizations to show the trends and patterns in GDP and related indicators over time.

Statistical analysis: Use statistical methods to analyze the data and identify patterns and relationships. Compute measures such as mean, median and perform regression analysis to determine the relationships between GDP and other economic indicators.

Interpretation: Interpret the results of the analysis and draw conclusions about the state of the economy. Identify any challenges or opportunities for growth and development and make recommendations for policymakers and stakeholders.

Python has many libraries for data analysis, including pandas, numpy, scipy, statsmodels, and scikit-learn. These libraries make it easy to perform GDP analysis and other types of data analysis using Python. With these tools, you can quickly analyze large datasets, identify trends and patterns, and make data-driven decisions to improve economic performance.

METHODOLOGY

Identifying the variables that are likely related to the GDP of that country, variables such as interest rates, literacy and birth rate. Splitting of data

Choosing the Machine learning model that best predicts the GDP and analyzes the factors that contributes the most to the GDP of that country

The model is developed, we have to determine the relationship between the variables and the independent variables

Split the data: Machine learning models requires a training dataset and a test dataset. The training dataset is used to build the model, while the test dataset is used to evaluate its accuracy. The data should be randomly split into a training set and a test set.

The data has been split, the next step is to selecting the appropriate independent variables and determining the optimal number of decision trees

Once the model has been built evaluate its accuracy using the test dataset. This can be done by comparing the predicted values of GDP with the actual values and calculating the error.

Finally, the results of the Random Forest Regression analysis or Linear Regression analysis can be used to make informed decisions about policy and investment. For example, if the analysis indicates that government spending has a strong positive relationship with GDP, policymakers may consider increasing spending as a way to stimulate economic growth.

MACHINE LEARNING MODELS

Linear Regression

Statistical model that models the relationship between the input features and the target variable as a linear equation. It is widely used due to its simplicity and interpretability. The understanding behind Linear regression is to identify a linear equation that best suits and predicts the value of one variable based on the other independent variable

Random Forest Regressor

Random Forest Regression is a machine learning technique that can be used for GDP analysis. It is a powerful algorithm that uses an ensemble of decision trees to predict the values of a dependent variable, such as GDP, based on a set of independent variables.

REQUIREMENTS AND IMPORTS

Pandas is an opensource Python package that is most widely used for data science/data analysis and machine learning tasks. It is built on top of another package named NumPy, which provides support for multi-dimensional arrays. As one of the most popular data wrangling packages

NumPy can be used to perform a wide variety of mathematical operations on arrays. It adds powerful data structures to Python that guarantee efficient calculations with arrays and matrices and it supplies an enormous library of high-level mathematical functions that operate on these arrays and matrices

Matplotlib is a library in Python that enables users to generate visualizations like histograms, scatter plots, bar charts, pie charts and much more.

Seaborn is a visualization library that is built on top of Matplotlib. It provides data visualizations that are typically more aesthetic and statistically sophisticated.

```
import numpy as np
import pandas as pd
import seaborn as sns
from matplotlib import pyplot as plt

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_squared_log_error
```

WEB SCRAPING

We are going to scrape the CIA The world factbook. Its an impressive website that stores detailed information of all countries.

First, we import everything required to help us import the data. Beautiful Soup is an impressive library that lets us scrape websites with ease in python.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re
import urllib.request, urllib.parse, urllib.error
from bs4 import BeautifulSoup
import ssl
```

This helps us ignore all securities and use beautiful soup to scrape the said website.

```
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

# Read the HTML from the URL and pass on to BeautifulSoup
url = 'https://www.cia.gov/library/publications/the-world-factbook/'
print("Opening the file connection...")
uh= urllib.request.urlopen(url, context=ctx)
print("HTTP status",uh.getcode())
html =uh.read().decode()
print(f"Reading done. Total {len(html)} characters read.")
```

```
Opening the file connection...
HTTP status 200
Reading done. Total 75036 characters read.
```

Parse the front-page HTML using BeautifulSoup and print it. Nicely.

```
soup = BeautifulSoup(html, 'html.parser')
print(soup.prettify())
```

```
Output exceeds the size limit. Open the full output data in a text editor
<!DOCTYPE html>
<!--[if lt IE 7]> <html class="no-js lt-ie9 lt-ie8 lt-ie7" lang="en"> <![endif]-->
<!--[if IE 7]>    <html class="no-js lt-ie9 lt-ie8" lang="en"> <![endif]-->
<!--[if IE 8]>    <html class="no-js lt-ie9" lang="en"> <![endif]-->
<!--[if gt IE 8]><!-->
<html class="no-js" lang="en" xml:lang="en" xmlns="http://www.w3.org/1999/xhtml">
 <!--<![endif]-->
 <head>
  <meta content="text/html; charset=utf-8" http-equiv="Content-Type"/>
  <link href="css/publications.css" rel="stylesheet" type="text/css"/>
  <link href="css/publications-detail.css" rel="stylesheet" type="text/css"/>
  <meta charset="utf-8"/>
  <meta content="IE=edge,chrome=1" http-equiv="X-UA-Compatible"/>
  <title>
   The World Factbook — Central Intelligence Agency
  </title>
  <meta content="" name="description"/>
```

We know how the website URLs work now, each countries data is arranged in the same way but under a different country code. So, in other words, if we get all the country codes we can iterate over all countries and scrape the required data.

```
country_codes=[]
country_names=[]
for tag in soup.find_all('option'):
    country_codes.append(tag.get('value')[5:7])
    country_names.append(tag.text)

temp=country_codes.pop(0) # To remove the first entry 'World'
temp=country_names.pop(0) # To remove the first entry 'World'
```

```
print('COUNTRY NAMES\n'+'-'*30)
for country in country_names[1:]:
    print(country,end=',')
print('\n\nCOUNTRY CODES\n'+'-'*30)
for country in country_codes[1:]:
    print(country,end=',')
```

```
COUNTRY NAMES
------------------------------
 Afghanistan , Akrotiri , Albania , Algeria , American Samoa , Andorra , Angola , Anguilla , Anta

COUNTRY CODES
------------------------------
af,ax,al,ag,ao,an,ao,av,av,ac,xq,ar,am,aa,at,zh,as,au,ai,bf,ba,um,bq,bb,bo,be,bh,bn,bd,bt,bl,bk,b
```

Now we add the Country Code to the URL and begin iteration.

```
# Base URL
urlbase = 'https://www.cia.gov/library/publications/the-world-factbook/geos/'
demographics1=[]
demographics2=[]
demographics3=[]
demographics4=[]
demographics5=[]

offset = len('65 years and over: ')

# Iterate over every country
for i in range(1,len(country_names)-1):
    country_html=country_codes[i]+'.html'
    url_to_get=urlbase+country_html
    # Read the HTML from the URL and pass on to BeautifulSoup
    html = urllib.request.urlopen(url_to_get, context=ctx).read()
    soup = BeautifulSoup(html, 'html.parser')

    txt=soup.get_text()
```

Save the DataFrame we made out of Scraped Data.

```
# Create a Pandas Excel writer using XlsxWriter as the engine.
writer = pd.ExcelWriter('Demo1.xlsx', engine='xlsxwriter')

# Convert the dataframe to an XlsxWriter Excel object.
df_demo.to_excel(writer, sheet_name='Countries of the World')

# Close the Pandas Excel writer and output the Excel file.
writer.save()
```

After CIA.gov was reworked, they now offer this archive as readymade data on their websites. This Dataset can also be found on Kaggle.

IMPORTING THE DATA

```
data = pd.read_csv('countries of the world.csv',decimal=',')
```

```
data.head()
```

	Country	Region	Population	Area (sq. mi.)	Pop. Density (per sq. mi.)	Coastline (coast/area ratio)	Net migration	Infant mortality (per 1000 births)	GDP ($ per capita)	Literacy (%)	Phones (per 1000)	Arable (%)
0	Afghanistan	ASIA (EX. NEAR EAST)	31056997	647500	48.0	0.00	23.06	163.07	700.0	36.0	3.2	12.13
1	Albania	EASTERN EUROPE	3581655	28748	124.6	1.26	-4.93	21.52	4500.0	86.5	71.2	21.09
2	Algeria	NORTHERN AFRICA	32930091	2381740	13.8	0.04	-0.39	31.00	6000.0	70.0	78.1	3.22
3	American Samoa	OCEANIA	57794	199	290.4	58.29	-20.71	9.27	8000.0	97.0	259.5	10.00
4	Andorra	WESTERN EUROPE	71201	468	152.1	0.00	6.60	4.05	19000.0	100.0	497.2	2.22

Climate	Birthrate	Deathrate	Agriculture	Industry	Service
1.0	46.60	20.34	0.380	0.240	0.380
3.0	15.11	5.22	0.232	0.188	0.579
1.0	17.14	4.61	0.101	0.600	0.298
2.0	22.46	3.27	NaN	NaN	NaN
3.0	8.71	6.25	NaN	NaN	NaN

DATA CLEANING AND EXPLORATION

First, we treat for null values, so it doesn't affect the outcome, we can't go ahead with operations if we don't treat for null values

```
data.isnull().sum()
```

```
Country                               0
Region                                0
Population                            0
Area (sq. mi.)                        0
Pop. Density (per sq. mi.)            0
Coastline (coast/area ratio)          0
Net migration                         3
Infant mortality (per 1000 births)    3
GDP ($ per capita)                    1
Literacy (%)                         18
Phones (per 1000)                     4
Arable (%)                            2
Crops (%)                             2
Other (%)                             2
Climate                              22
Birthrate                             3
Deathrate                             4
Agriculture                          15
Industry                             16
Service                              15
dtype: int64
```

```
data.describe(include = 'all')
```

	Country	Region	Population	Area (sq. mi.)	Pop. Density (per sq. mi.)	Coastline (coast/area ratio)	Net migration	Infant mortality (per 1000 births)	GDP ($ per capita)	Literacy (%)
count	227	227	2.270000e+02	2.270000e+02	227.000000	227.000000	227.000000	227.000000	227.000000	227.000000
unique	227	11	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
top	Afghanistan	SUB-SAHARAN AFRICA	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
freq	1	51	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
mean	NaN	NaN	2.874028e+07	5.982270e+05	379.047137	21.165330	0.035903	35.285463	9673.568282	83.588106
std	NaN	NaN	1.178913e+08	1.790282e+06	1660.185825	72.286863	4.856794	35.220671	10029.871625	19.315834
min	NaN	NaN	7.026000e+03	2.000000e+00	0.000000	0.000000	-20.990000	2.290000	500.000000	17.600000
25%	NaN	NaN	4.376240e+05	4.647500e+03	29.150000	0.100000	-0.905000	8.215000	1900.000000	75.300000
50%	NaN	NaN	4.786994e+06	8.660000e+04	78.800000	0.730000	0.000000	20.970000	5600.000000	92.600000
75%	NaN	NaN	1.749777e+07	4.418110e+05	190.150000	10.345000	0.980000	55.335000	15700.000000	98.000000
max	NaN	NaN	1.313974e+09	1.707520e+07	16271.500000	870.660000	23.060000	191.190000	55100.000000	100.000000

There are many ways of treating null values, but we are going to just fill it with median values that is sorted by the region. The reason why we are choosing region/climate is because geologically close countries are often similar in many aspects.

We also use climate to navigate because it is categorical in nature.

```
data.groupby('Region')[['GDP ($ per capita)','Literacy (%)','Agriculture']].median()
```

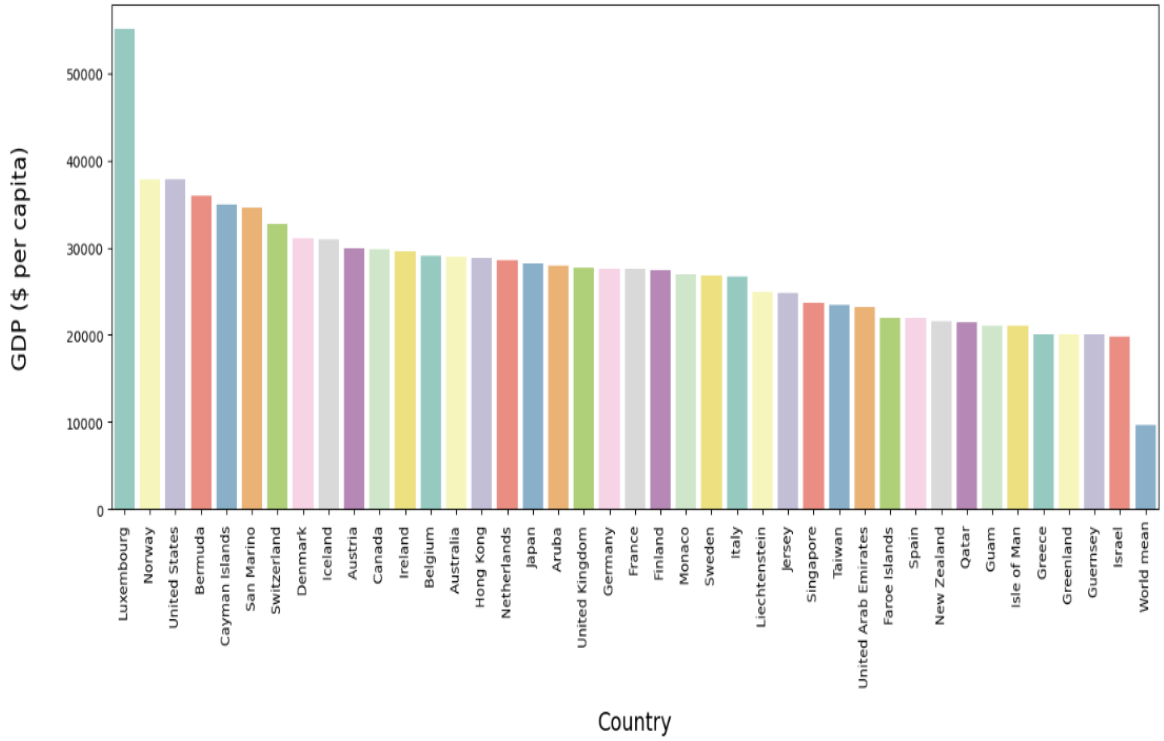
Region	GDP ($ per capita)	Literacy (%)	Agriculture
ASIA (EX. NEAR EAST)	3450.0	90.60	0.1610
BALTICS	11400.0	99.80	0.0400
C.W. OF IND. STATES	3450.0	99.05	0.1980
EASTERN EUROPE	9100.0	98.60	0.0815
LATIN AMER. & CARIB	6300.0	94.05	0.0700
NEAR EAST	9250.0	83.00	0.0350
NORTHERN AFRICA	6000.0	70.00	0.1320
NORTHERN AMERICA	29800.0	97.50	0.0100
OCEANIA	5000.0	95.00	0.1505
SUB-SAHARAN AFRICA	1300.0	62.95	0.2760
WESTERN EUROPE	27200.0	99.00	0.0220

```
data.groupby('Region')[['GDP ($ per capita)','Literacy (%)','Agriculture']].median()
for col in data.columns.values:
    if data[col].isnull().sum() == 0:
        continue
    if col == 'Climate':
        guess_values = data.groupby('Region')['Climate'].apply(lambda x:x.mode().max())
    else:
        guess_values = data.groupby('Region')[col].median()
    for region in data['Region'].unique():
        data[col].loc[(data[col].isnull())&(data['Region']==region)] = guess_values[region]
```

Let’s have a look at the top 40 countries with the highest GDP per capita. Of course, we might only need the first 10-15 of them.

```
fig, ax = plt.subplots(figsize=(16,6))
#ax = fig.add_subplot(111)
top_gdp_countries = df.sort_values('GDP ($ per capita)',ascending=False).head(40)
mean = pd.DataFrame({'Country':['World mean'], 'GDP ($ per capita)':[df['GDP ($ per capita)'].mean()]})
gdps = pd.concat([top_gdp_countries[['Country','GDP ($ per capita)']],mean],ignore_index=True)

sns.barplot(x='Country',y='GDP ($ per capita)',data=gdps, palette='Set3')
ax.set_xlabel(ax.get_xlabel(),labelpad=15)
ax.set_ylabel(ax.get_ylabel(),labelpad=30)
ax.xaxis.label.set_fontsize(16)
ax.yaxis.label.set_fontsize(16)
plt.xticks(rotation=90)
plt.show()
```

As we can see, Luxembourg is far ahead than the next few countries. The world mean was added just for reference.

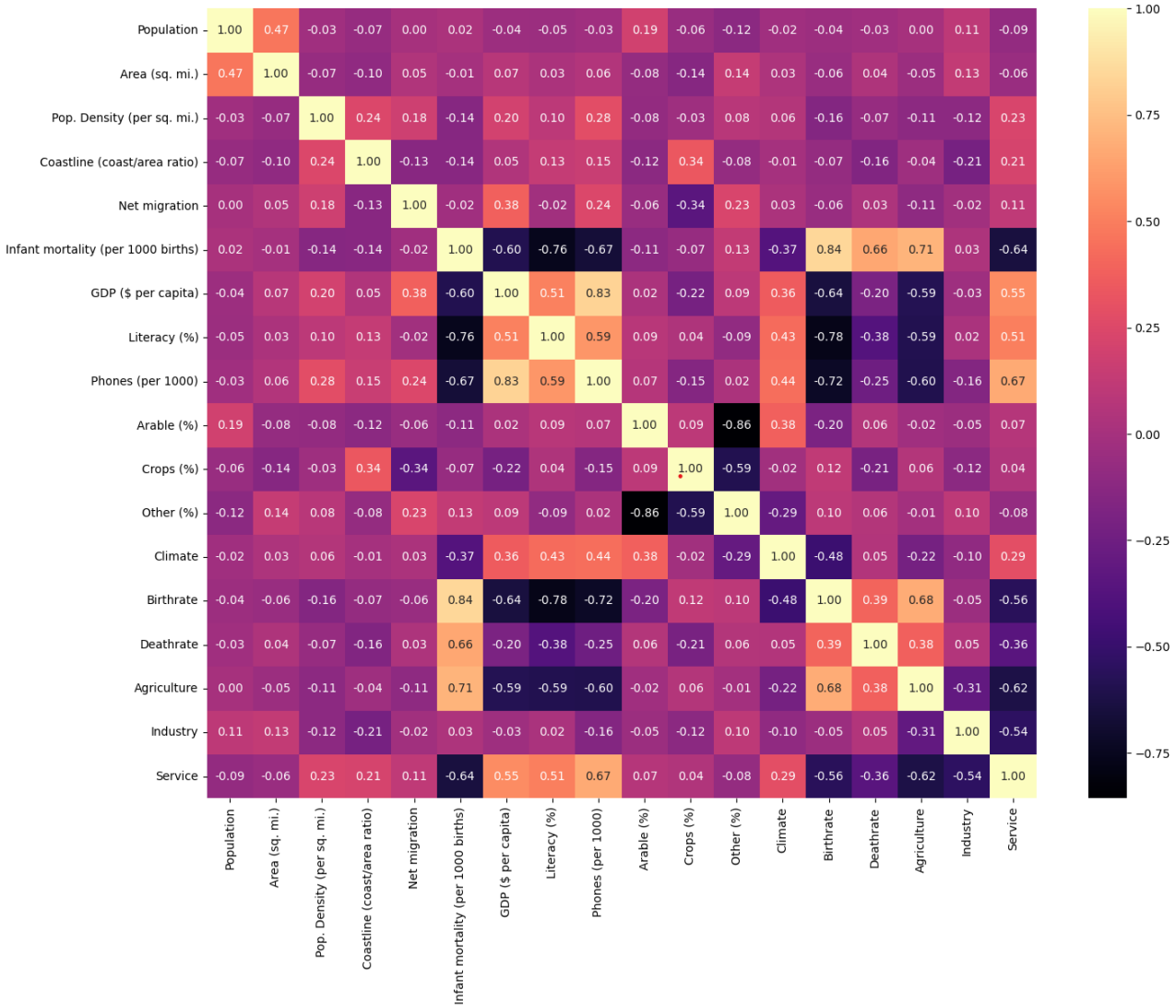


CORRELATION BETWEEN EACH FACTOR OF THE DATA

Let's check if the factors have correlation with each other.

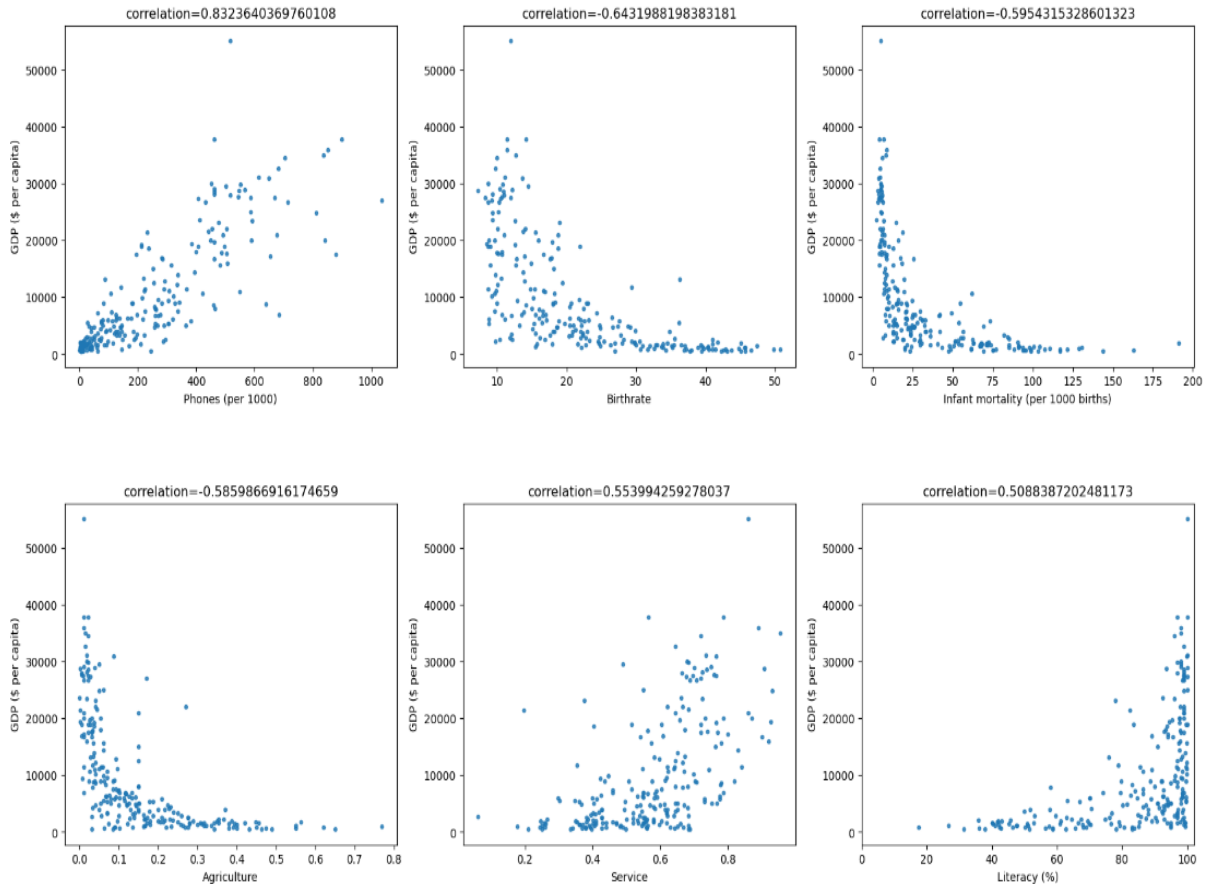
```
plt.figure(figsize=(16,12))
sns.heatmap(data=data.iloc[:,2:].corr(),annot=True,fmt='.2f',cmap='magma')
plt.show()
```

With the help of this Heatmap we can see the factors that are majorly affecting GDP per capita. Most of the results strongly agree with common sense.



We pick a few columns that have strong correlation and try to see what kind of relation it has using a scatter plot.

```
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(20,12))
plt.subplots_adjust(hspace=0.4)

corr_to_gdp = pd.Series()
for col in data.columns.values[2:]:
    if ((col!='GDP ($ per capita)')&(col!='Climate')):
        corr_to_gdp[col] = data['GDP ($ per capita)'].corr(data[col])
abs_corr_to_gdp = corr_to_gdp.abs().sort_values(ascending=False)
corr_to_gdp = corr_to_gdp.loc[abs_corr_to_gdp.index]

for i in range(2):
    for j in range(3):
        sns.regplot(x=corr_to_gdp.index.values[i*3+j], y='GDP ($ per capita)', data=data,
                    ax=axes[i,j], fit_reg=False, marker='.')
        title = 'correlation='+str(corr_to_gdp[i*3+j])
        axes[i,j].set_title(title)
axes[1,2].set_xlim(0,102)
plt.show()
```



Let’s Filter out countries with low Birthrate and low GDP per capita. Since by correlation we can see that Low Birthrate means High GDP. Treating exceptions can give us a better idea of correlation.

```
data.loc[(data['Birthrate']<14)&(data['GDP ($ per capita)']<10000)].head()
```

	Country	Region	Population	Area (sq. mi.)	Pop. Density (per sq. mi.)	Coastline (coast/area ratio)	Net migration	Infant mortality (per 1000 births)	GDP ($ per capita)	Literacy (%)	Phones (per 1000)	Arable (%)
9	Armenia	C.W. OF IND. STATES	2976372	29800	99.9	0.00	-6.47	23.28	3500.0	98.6	195.7	17.55
18	Belarus	C.W. OF IND. STATES	10293011	207600	49.6	0.00	2.54	13.37	6100.0	99.6	319.1	29.55
25	Bosnia & Herzegovina	EASTERN EUROPE	4498976	51129	88.0	0.04	0.31	21.05	6100.0	NaN	215.4	13.60
30	Bulgaria	EASTERN EUROPE	7385367	110910	66.6	0.32	-4.58	20.55	7600.0	98.6	336.3	40.02
42	China	ASIA (EX. NEAR EAST)	1313973713	9596960	136.9	0.15	-0.40	24.18	5000.0	90.9	266.7	15.40

MODELLING TRAINING AND TESTING

We will use the labels of the 2 categorical features we have been given.

```
LE = LabelEncoder()
data['Region_label'] = LE.fit_transform(data['Region'])
data['Climate_label'] = LE.fit_transform(data['Climate'])
data.head()
```

	Country	Region	Population	Area (sq. mi.)	Pop. Density (per sq. mi.)	Coastline (coast/area ratio)	Net migration	Infant mortality (per 1000 births)	GDP ($ per capita)	Literacy (%)	...	Crops (%)	Other (%)
0	Afghanistan	ASIA (EX. NEAR EAST)	31056997	647500	48.0	0.00	23.06	163.07	700.0	36.0	...	0.22	87.65
1	Albania	EASTERN EUROPE	3581655	28748	124.6	1.26	-4.93	21.52	4500.0	86.5	...	4.42	74.49
2	Algeria	NORTHERN AFRICA	32930091	2381740	13.8	0.04	-0.39	31.00	6000.0	70.0	...	0.25	96.53
3	American Samoa	OCEANIA	57794	199	290.4	58.29	-20.71	9.27	8000.0	97.0	...	15.00	75.00
4	Andorra	WESTERN EUROPE	71201	468	152.1	0.00	6.60	4.05	19000.0	100.0	...	0.00	97.78

5 rows × 22 columns

Prepare the Data so Machine learning models can be applied on them.

```
train, test = train_test_split(data, test_size=0.3, shuffle=True)
training_features = ['Population', 'Area (sq. mi.)',
       'Pop. Density (per sq. mi.)', 'Coastline (coast/area ratio)',
       'Net migration', 'Infant mortality (per 1000 births)',
       'Literacy (%)', 'Phones (per 1000)',
       'Arable (%)', 'Crops (%)', 'Other (%)', 'Birthrate',
       'Deathrate', 'Agriculture', 'Industry', 'Service', 'Region_label',
       'Climate_label','Service']
target = 'GDP ($ per capita)'
train_X = train[training_features]
train_Y = train[target]
test_X = test[training_features]
test_Y = test[target]
```

APPLYING MODELS

Let’s start with Linear regression model. We will use RMSE for accuracy.

Linear regression model

```
model = LinearRegression()
model.fit(train_X, train_Y)
train_pred_Y = model.predict(train_X)
test_pred_Y = model.predict(test_X)
model.score(train_X, train_Y)
```

```
0.7517397877080385
```

```
model = LinearRegression()
model.fit(train_X, train_Y)
train_pred_Y = model.predict(train_X)
test_pred_Y = model.predict(test_X)
train_pred_Y = pd.Series(train_pred_Y.clip(0, train_pred_Y.max()), index=train_Y.index)
test_pred_Y = pd.Series(test_pred_Y.clip(0, test_pred_Y.max()), index=test_Y.index)

rmse_train = np.sqrt(mean_squared_error(train_pred_Y, train_Y))
msle_train = mean_squared_log_error(train_pred_Y, train_Y)
rmse_test = np.sqrt(mean_squared_error(test_pred_Y, test_Y))
msle_test = mean_squared_log_error(test_pred_Y, test_Y)

print('rmse_train:',rmse_train,'msle_train:',msle_train)
print('rmse_test:',rmse_test,'msle_test:',msle_test)
```

```
rmse_train: 4324.194159440362 msle_train: 5.173766428522456
rmse_test: 5853.54522489157 msle_test: 4.641313166432838
```

Countries since we are aware that our target variable may not necessarily be linear or continuous, it might be worth our time to try other models as well

For example, Random Forest

Random Forest Model

```
model = RandomForestRegressor(n_estimators = 50,
                              max_depth = 6,
                              min_weight_fraction_leaf = 0.05,
                              max_features = 0.8,
                              random_state = 42)
model.fit(train_X, train_Y)
train_pred_Y = model.predict(train_X)
test_pred_Y = model.predict(test_X)
model.score(train_X, train_Y)
```

```
0.9143715423132895
```

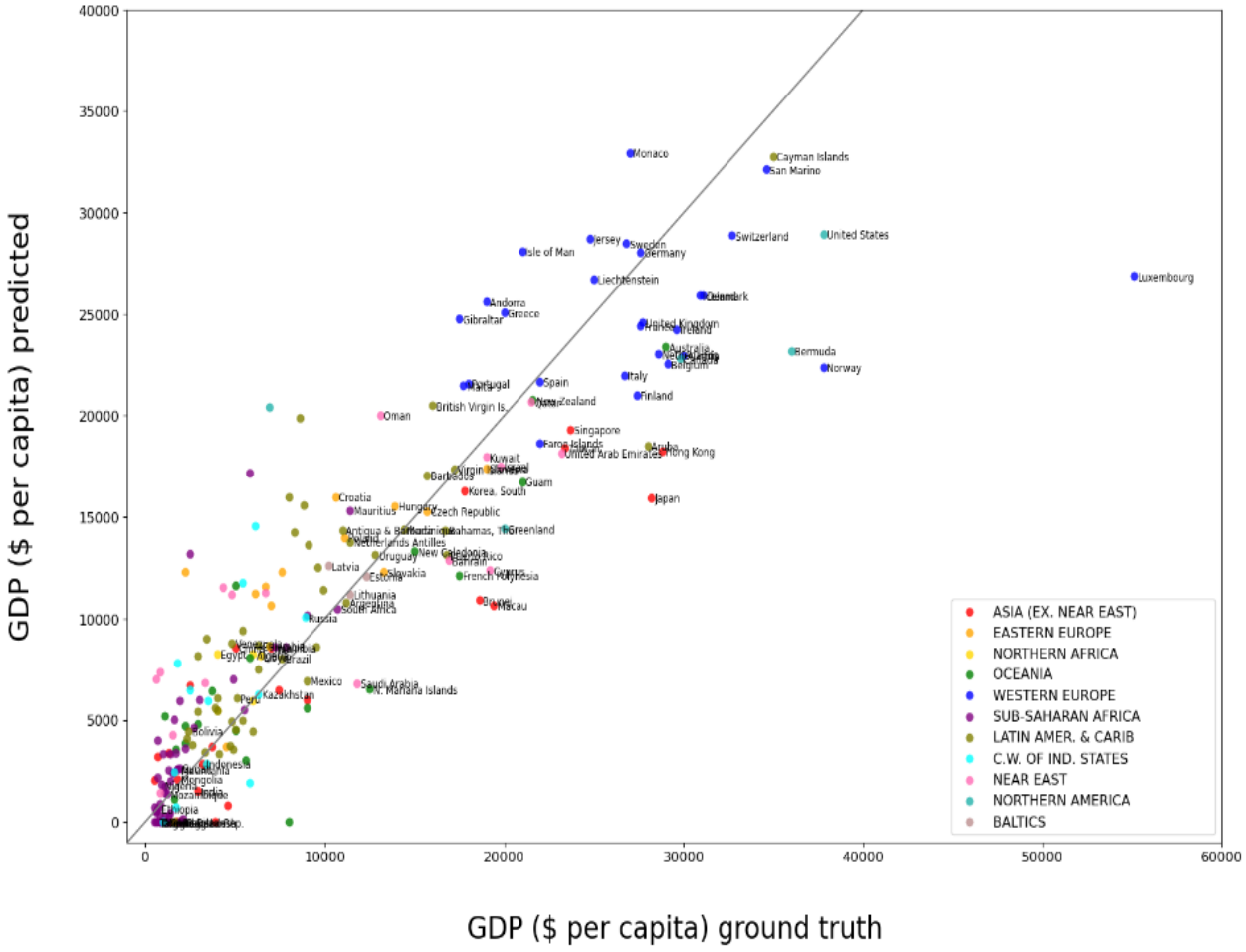
```
model = RandomForestRegressor(n_estimators = 50,
                              max_depth = 6,
                              min_weight_fraction_leaf = 0.05,
                              max_features = 0.8,
                              random_state = 42)
model.fit(train_X, train_Y)
train_pred_Y = model.predict(train_X)
test_pred_Y = model.predict(test_X)
train_pred_Y = pd.Series(train_pred_Y.clip(0, train_pred_Y.max()), index=train_Y.index)
test_pred_Y = pd.Series(test_pred_Y.clip(0, test_pred_Y.max()), index=test_Y.index)

rmse_train = np.sqrt(mean_squared_error(train_pred_Y, train_Y))
msle_train = mean_squared_log_error(train_pred_Y, train_Y)
rmse_test = np.sqrt(mean_squared_error(test_pred_Y, test_Y))
msle_test = mean_squared_log_error(test_pred_Y, test_Y)

print('rmse_train:',rmse_train,'msle_train:',msle_train)
print('rmse_test:',rmse_test,'msle_test:',msle_test)
```

```
rmse_train: 3497.327429395617 msle_train: 0.19064228534026212
rmse_test: 2566.4853229574414 msle_test: 0.18371884063692645
```

VISUALIZATION

```
train_test_Y = train_Y.append(test_Y)
train_test_pred_Y = train_pred_Y.append(test_pred_Y)

data_shuffled = data.loc[train_test_Y.index]
label = data_shuffled['Country']

colors = {'ASIA (EX. NEAR EAST)         ':'red',
          'EASTERN EUROPE                     ':'orange',
          'NORTHERN AFRICA                    ':'gold',
          'OCEANIA                            ':'green',
          'WESTERN EUROPE                     ':'blue',
          'SUB-SAHARAN AFRICA                 ':'purple',
          'LATIN AMER. & CARIB    ':'olive',
          'C.W. OF IND. STATES ':'cyan',
          'NEAR EAST                          ':'hotpink',
          'NORTHERN AMERICA                   ':'lightseagreen',
          'BALTICS                            ':'rosybrown'}

for region, color in colors.items():
    X = train_test_Y.loc[data_shuffled['Region']==region]
    Y = train_test_pred_Y.loc[data_shuffled['Region']==region]
    ax = sns.regplot(x=X, y=Y, marker='.', fit_reg=False, color=color, scatter_kws={'s':200, 'linewidths':0}
plt.legend(loc=4,prop={'size': 12})

ax.set_xlabel('GDP ($ per capita) ground truth',labelpad=40)
ax.set_ylabel('GDP ($ per capita) predicted',labelpad=40)
ax.xaxis.label.set_fontsize(24)
ax.yaxis.label.set_fontsize(24)
ax.tick_params(labelsize=12)

x = np.linspace(-1000,50000,100) # 100 linearly spaced numbers
y = x
plt.plot(x,y,c='gray')

plt.xlim(-1000,60000)
plt.ylim(-1000,40000)

for i in range(0,train_test_Y.shape[0]):
    if((data_shuffled['Area (sq. mi.)'].iloc[i]>8e5) |
       (data_shuffled['Population'].iloc[i]>1e8) |
       (data_shuffled['GDP ($ per capita)'].iloc[i]>10000)):
        plt.text(train_test_Y.iloc[i]+200, train_test_pred_Y.iloc[i]-200, label.iloc[i], size='small')
```

To display how well our model is doing, we have made a graph of datapoints of the predicted GDP vs The Actual GDP Since most of the countries are gathering around the diagonal line, we can safely say that our model is working well.



Linear Regression vs Random Forest

Coefficients

Random Forest

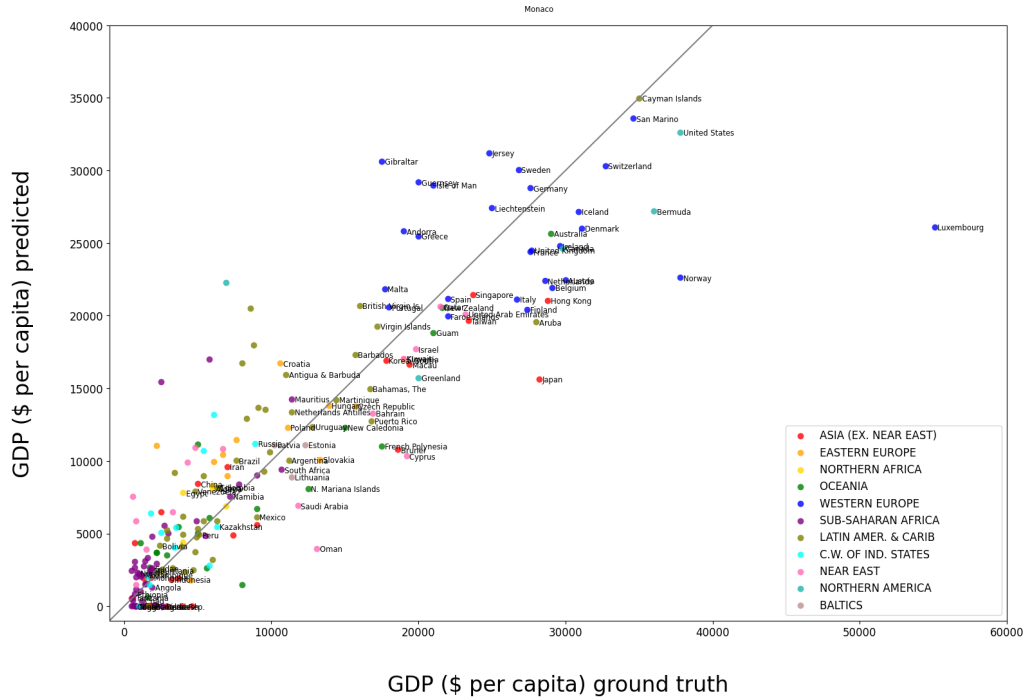
category	coef
Phones (per 1000)	0.492913
Infant mortality (per 1000 births)	0.267029
Agriculture	0.098156
Birthrate	0.045650
Net migration	0.026203
Crops (%)	0.013125
Literacy (%)	0.011644
Service	0.006796
Industry	0.005390
Area (sq. mi.)	0.005324
Region_label	0.005234
Coastline (coast/area ratio)	0.005207
Deathrate	0.004519
Other (%)	0.003935
Pop. Density (per sq. mi.)	0.003430
Service	0.001972
Arable (%)	0.001960
Population	0.000892
Climate_label	0.000621

Linear regression

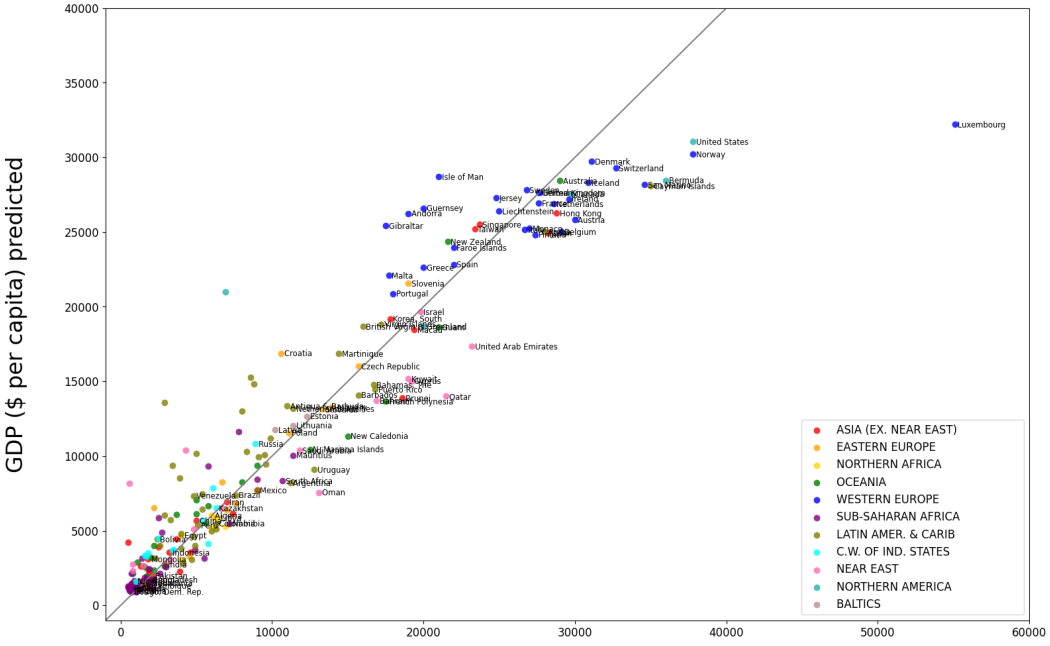
category	coef
Other (%)	5203.675034
Arable (%)	5189.616838
Crops (%)	5158.085353
Region_label	572.196636
Net migration	423.924815
Industry	309.690192
Phones (per 1000)	30.126210
Deathrate	15.915836
Literacy (%)	8.679271
Pop. Density (per sq. mi.)	0.304958
Population	-0.000001
Area (sq. mi.)	-0.000046
Infant mortality (per 1000 births)	-12.395022
Coastline (coast/area ratio)	-27.675170
Birthrate	-158.046292
Climate_label	-267.874291
Service	-1774.867223
Service	-1774.867223
Agriculture	-4348.334962

Prediction To Truth Plot

Linear



Random Forest

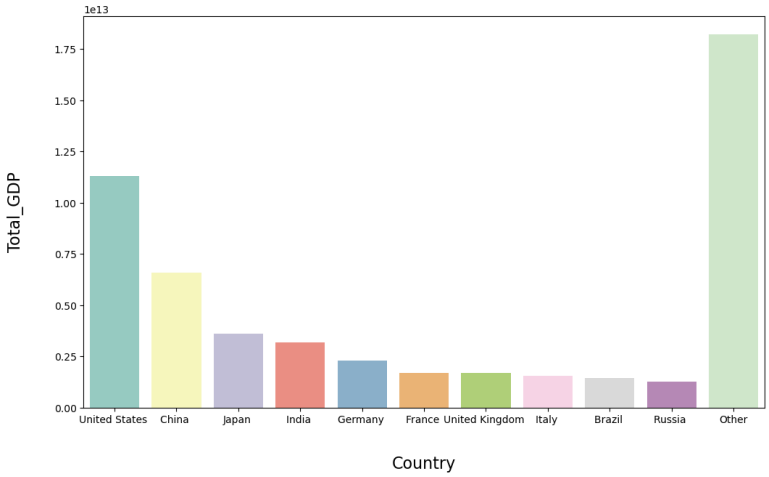


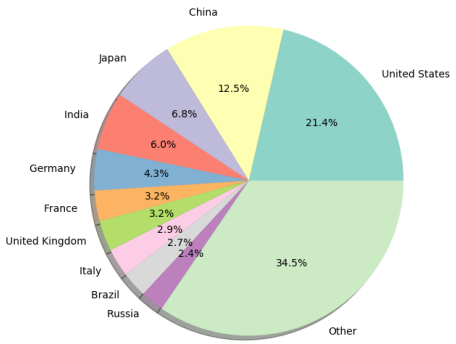
Total GDP

For further research, we can take Total GDP. We simply multiple the GDP per capita with the population of each country.

We also create the “Other” column for comparison

```
data['Total_GDP ($)'] = data['GDP ($ per capita)'] * data['Population']
#plt.figure(figsize=(16,6))
top_gdp_countries = data.sort_values('Total_GDP ($)',ascending=False).head(10)
other = pd.DataFrame({'Country':['Other'], 'Total_GDP ($)':[data['Total_GDP ($)'].sum() - top_gdp_countries
gdps = pd.concat([top_gdp_countries[['Country','Total_GDP ($)']],other],ignore_index=True)

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20,7),gridspec_kw = {'width_ratios':[2,1]})
sns.barplot(x='Country',y='Total_GDP ($)',data=gdps,ax=axes[0],palette='Set3')
axes[0].set_xlabel('Country',labelpad=30,fontsize=16)
axes[0].set_ylabel('Total_GDP',labelpad=30,fontsize=16)

colors = sns.color_palette("Set3", gdps.shape[0]).as_hex()
axes[1].pie(gdps['Total_GDP ($)'], labels=gdps['Country'],colors=colors,autopct='%1.1f%%',shadow=True)
axes[1].axis('equal')
plt.show()
```





This Ranks Countries according to GDP per capita and GDP, here we are checking by how much the countries ranking change when we compare them.

```
Rank1 = data[['Country','Total_GDP ($)']].sort_values('Total_GDP ($)', ascending=False).reset_index()
Rank2 = data[['Country','GDP ($ per capita)']].sort_values('GDP ($ per capita)', ascending=False).reset_inde
Rank1 = pd.Series(Rank1.index.values+1, index=Rank1.Country)
Rank2 = pd.Series(Rank2.index.values+1, index=Rank2.Country)
Rank_change = (Rank2-Rank1).sort_values(ascending=False)
print('rank of total GDP - rank of GDP per capita:')
Rank_change.loc[top_gdp_countries.Country]
```

```
rank of total GDP - rank of GDP per capita:


Country
United States       1
China             118
Japan              14
India             146
Germany            15
France             15
United Kingdom     12
Italy              17
Brazil             84
Russia             75
dtype: int64
```

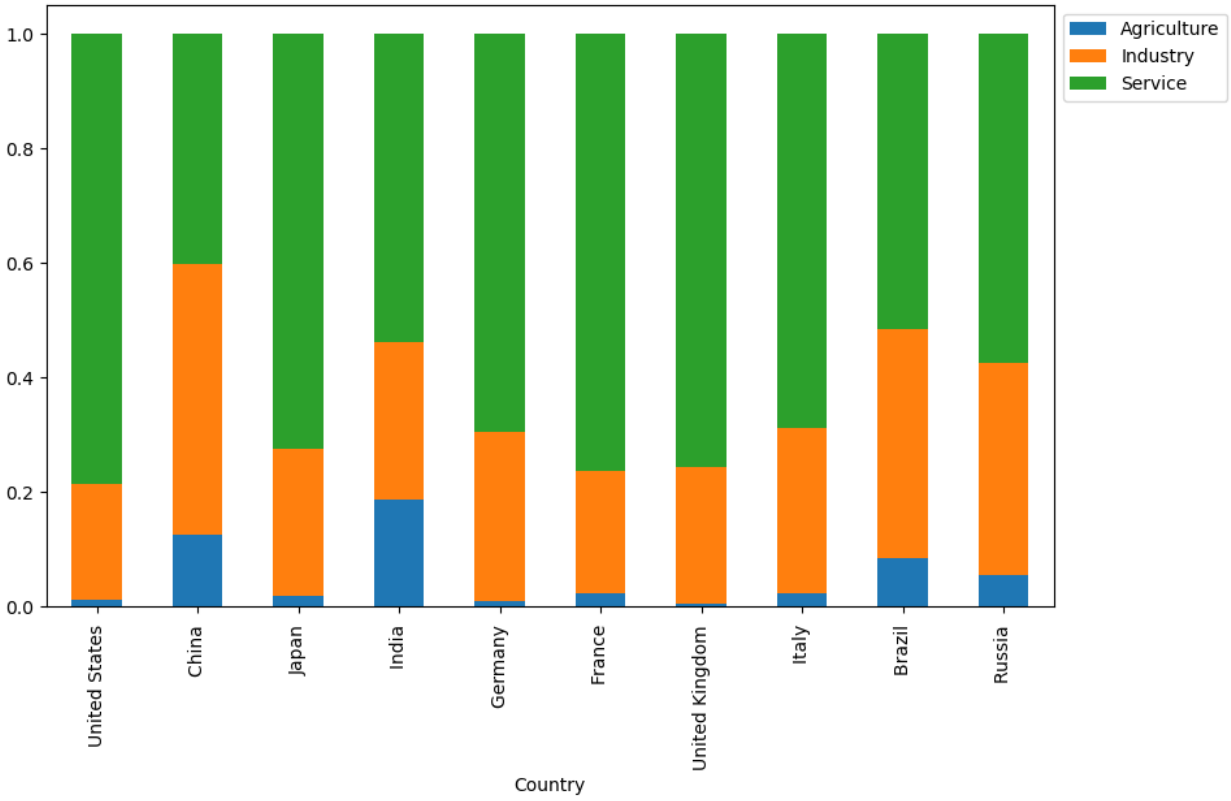
Let's check the correlation between all columns to total GDP and as expected, Population has high correlation

```
corr_to_gdp = pd.Series()
for col in data.columns.values[2:]:
    if ((col!='Total_GDP ($)')&(col!='Climate')&(col!='GDP ($ per capita)')):
        corr_to_gdp[col] = data['Total_GDP ($)'].corr(data[col])
abs_corr_to_gdp = corr_to_gdp.abs().sort_values(ascending=False)
corr_to_gdp = corr_to_gdp.loc[abs_corr_to_gdp.index]
print(corr_to_gdp)
```

```
Population                          0.639528
Area (sq. mi.)                      0.556396
Phones (per 1000)                   0.233484
Birthrate                          -0.166889
Agriculture                        -0.139516
Arable (%)                          0.129928
Climate_label                       0.125791
Infant mortality (per 1000 births) -0.122076
Literacy (%)                        0.099417
Service                             0.085096
Region_label                       -0.079745
Crops (%)                          -0.077078
Coastline (coast/area ratio)       -0.065211
Other (%)                          -0.064882
Net migration                       0.054632
Industry                            0.050399
Deathrate                          -0.035820
Pop. Density (per sq. mi.)         -0.028487
dtype: float64
```

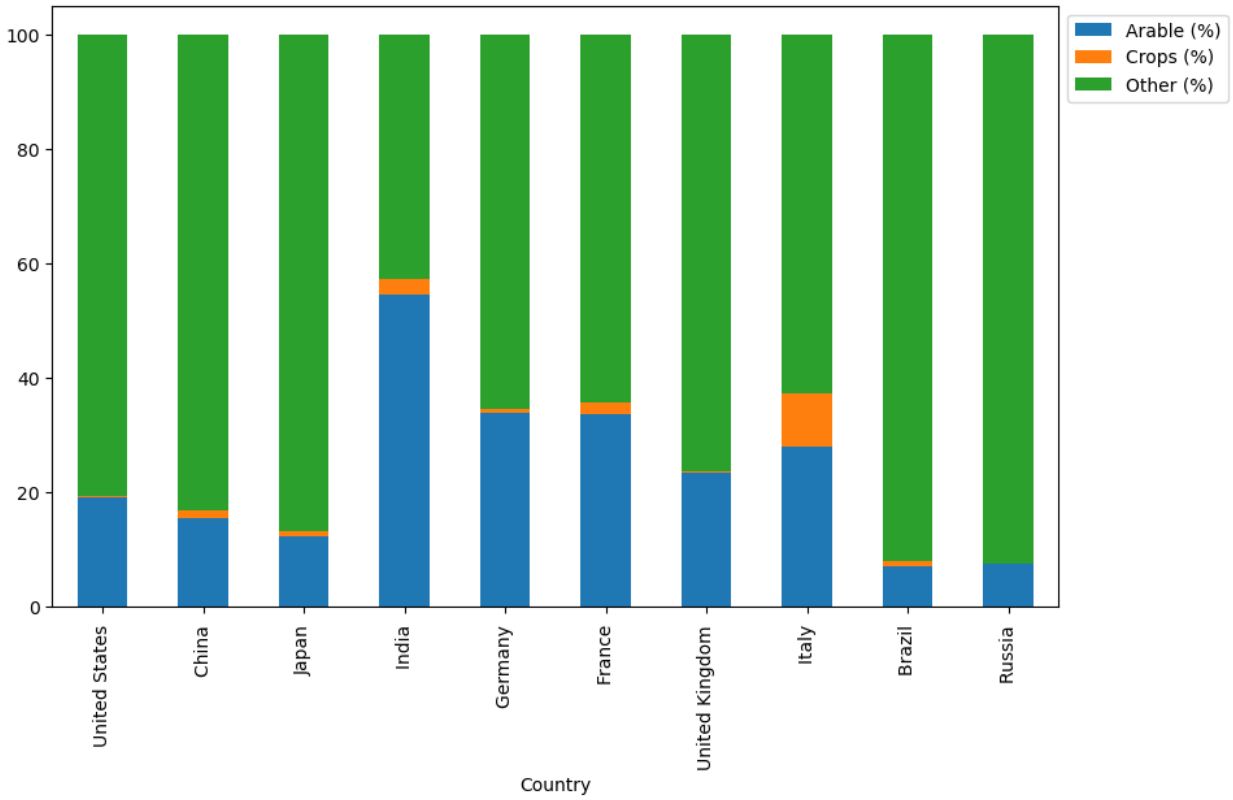
Finally, let’s compare what kind of "economic structure" these top countries follow.

```
plot_data = top_gdp_countries.head(10)[['Country','Agriculture', 'Industry', 'Service']]
plot_data = plot_data.set_index('Country')
ax = plot_data.plot.bar(stacked=True,figsize=(10,6))
ax.legend(bbox_to_anchor=(1, 1))
plt.show()
```



While we are at it, let's also check what kind of land usage these countries have

```
plot_data = top_gdp_countries[['Country','Arable (%)', 'Crops (%)', 'Other (%)']]
plot_data = plot_data.set_index('Country')
ax = plot_data.plot.bar(stacked=True,figsize=(10,6))
ax.legend(bbox_to_anchor=(1, 1))
plt.show()
```



CONCLUSION

Linear Regression

ACCURACY	RMSE TRAIN	RMSE TEST	MSLE TRAIN	MSLE TEST
78%	4324.19415	5853.54522	5.17373	4.64131

Random Forest Regression

ACCURACY	RMSE TRAIN	RMSE TEST	MSLE TRAIN	MSLE TEST
91%	3497.32745	2566.4854	0.19864	0.18371

The top factors affecting the total GDP are population and area and followed by many other that have been found mostly correlated to the GDP per capita are agriculture, birthrate and climate. The least contributing factors consist of Deathrate and migration. Top GDP countries include United states of America, China, India, Japan and Germany. Agriculture, industry and service area the three most important factor affecting the GDP of the top countries and their area that is land is majorly contributed by crops.

Country with the highest GDP is Luxembourg followed by the likes of United states of America, Denmark and Switzerland.

Since the predicted graph of the GDP of countries are situated around the diagonal shows that our model is safe and we can conclude that our machine learning model is justifiable.